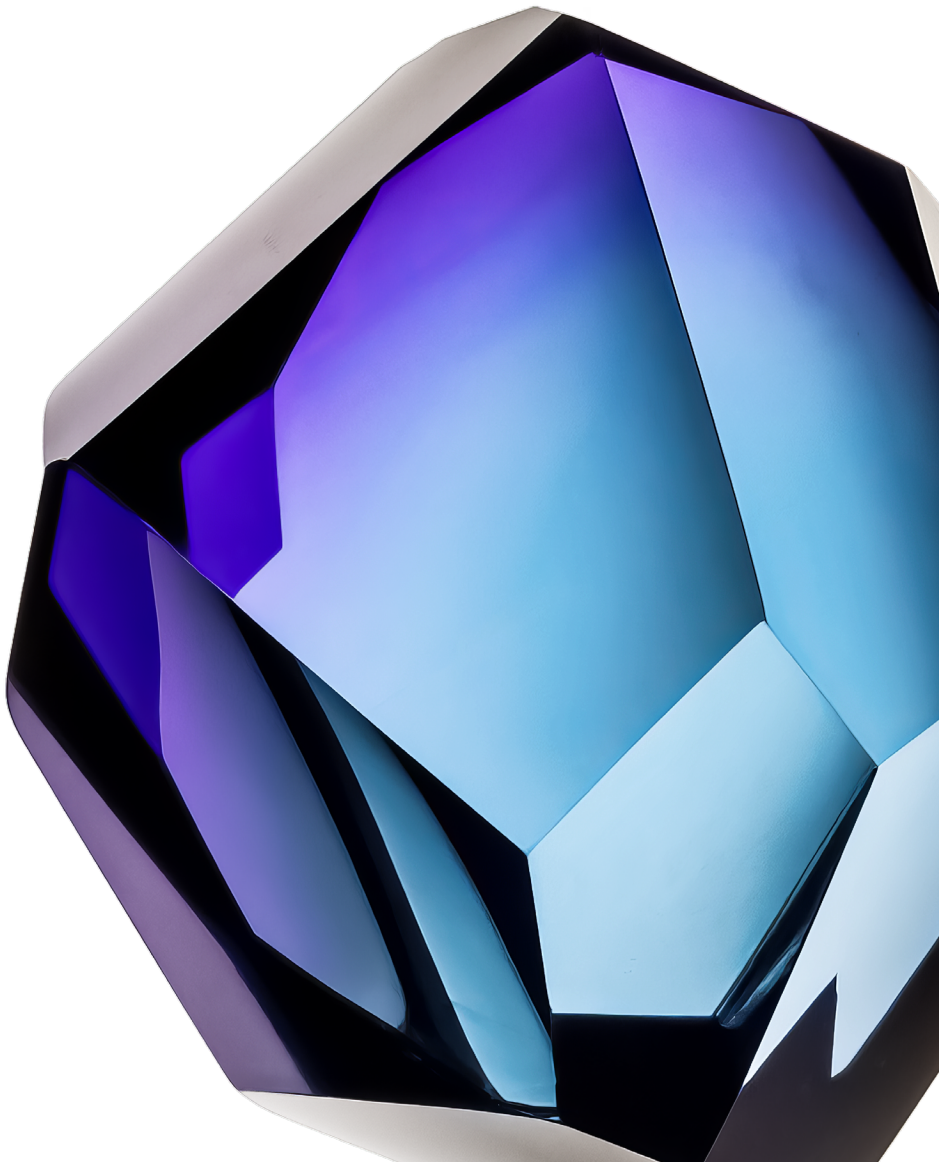
提示词工程

作者：Lee Boonstra



Google

致谢

审阅者与贡献者

Michael Sherman

Yuan Cao

Erick Armbrust

Anant Nawalgaria

Antonio Gulli

Simone Cammel

策划与编辑

Antonio Gulli

Anant Nawalgaria

Grace Mollison

技术撰稿

Joey Haymaker

设计

Michael Lanning

目录

引言 4
提示词工程 4
大语言模型的输出配置 5
输出长度 5
采样控制 5
温度 5
Top-K 与 top-P 5
综合运用 6
提示技巧 6
通用提示 / 零样本 7
单样本与少样本 7
系统提示、上下文提示与角色提示 9
系统提示 9
角色提示 11
上下文提示 12
退后一步提示 13
思维链（CoT） 15
自洽性 16
思维树（ToT） 18
ReAct（推理与行动） 19
自动提示词工程 21
代码提示 21
用于编写代码的提示词 22
用于解释代码的提示词 23
用于翻译代码的提示词 24
用于调试和审查代码的提示词 26
多模态提示呢? 30
最佳实践 30
提供示例 30
简洁设计 30
明确输出的具体要求 30
优先使用指令，而非约束 31
控制最大词元长度 31
在提示词中使用变量 32
尝试不同的输入格式和写作风格 32
少样本分类任务要打乱类别顺序 32
适应模型更新 32
尝试不同的输出格式 33
与其他提示词工程师一起实验 33
思维链的最佳实践 33
记录各种提示词尝试 33
总结 34
尾注 35

> 你不需要成为数据科学家或机器学习工程师——每个人都能编写提示词。

引言

在思考大语言模型的输入与输出时，文本提示词（有时还会搭配图像提示等其他模态）就是模型用来预测特定输出的输入。你不需要成为数据科学家或机器学习工程师——每个人都能编写提示词。然而，要写出最有效的提示词可能并不简单。提示词的许多方面都会影响其效果：你所使用的模型、模型的训练数据、模型配置，以及你的用词、风格与语气、结构和上下文，全都很重要。因此，提示词工程是一个迭代的过程。不充分的提示词可能导致含糊、不准确的回答，并会削弱模型给出有意义输出的能力。

提示词工程

当你与 Gemini 聊天机器人[1]对话时，其实就是在编写提示词。不过，本白皮书聚焦于为 Vertex AI 中的 Gemini 模型编写提示词，或通过 API 编写提示词，因为只有直接向模型发起提示时，你才能访问温度（temperature）等配置项。

本白皮书将详细讨论提示词工程。我们会介绍各种提示技巧，帮助你快速上手，并分享成为提示词专家的技巧与最佳实践。我们还会讨论在编写提示词时可能遇到的一些挑战。

请记住大语言模型的工作原理：它是一个预测引擎。模型以顺序文本作为输入，然后根据其训练数据预测下一个词元应该是什么。大语言模型会反复执行这一操作：把此前预测出的词元追加到顺序文本的末尾，以此预测下一个词元。下一个词元的预测，取决于已有词元之间的关系，以及大语言模型在训练过程中见过的内容。

当你编写提示词时，实际上是在为大语言模型创造条件，让它预测出正确的词元序列。提示词工程就是设计高质量的提示词、引导大语言模型产出准确输出的过程。这一过程包括反复调试以找到最佳提示词、优化提示词长度，以及结合具体任务评估提示词的写作风格与结构。在自然语言处理和大语言模型的语境中，提示词就是提供给模型的输入，用来生成回答或预测。

这些提示词可用于完成各类理解与生成任务，例如文本摘要、信息抽取、问答、文本分类、语言或代码翻译、代码生成，以及代码文档撰写或推理。

你还可以参考 Google 的提示指南[2,3]，其中包含简单而有效的提示示例。

做提示词工程时，你首先要选择一个模型。无论你使用的是 Vertex AI 中的 Gemini 语言模型、GPT、Claude，还是 Gemma、LLaMA 这类开源模型，提示词都可能需要针对具体模型进行优化。

除了提示词本身，你还需要调试大语言模型的各种配置。

大语言模型的输出配置

选定模型后，你需要确定模型配置。大多数大语言模型都提供多种配置选项来控制其输出。有效的提示词工程需要针对你的任务把这些配置调到最优。

输出长度

一项重要的配置是回答中要生成的词元数量。生成更多词元需要大语言模型进行更多计算，会导致能耗更高、响应可能更慢，成本也更高。

缩短大语言模型的输出长度，并不会让它生成的文本在风格或措辞上变得更简洁；它只是在达到上限后停止预测更多词元。如果你的场景需要较短的输出，可能还需要据此对提示词进行专门设计。

限制输出长度对某些提示技巧尤为重要，例如 ReAct，因为在这种技巧下，大语言模型在给出你想要的回答之后仍会继续输出无用的词元。

采样控制

大语言模型并不是直接预测出某一个词元，而是预测下一个词元可能是什么的概率，词表中的每个词元都会得到一个概率值。随后对这些词元概率进行采样，以决定实际生成的下一个词元。温度、top-K 和 top-P 是最常见的配置项，它们决定如何处理预测出的词元概率，从而选出最终的一个输出词元。

温度

温度控制词元选择的随机程度。较低的温度适合期望得到更确定回答的提示词，较高的温度则会带来更多样、更出人意料的结果。温度为 0（贪婪解码）时结果是确定的：始终选择概率最高的词元（但要注意，如果两个词元并列拥有最高预测概率，那么取决于打破平局的实现方式，温度为 0 时你也未必总能得到相同的输出）。

温度接近最大值时，输出往往更加随机。随着温度越来越高，所有词元成为下一个预测词元的可能性会趋于相同。

Gemini 的温度控制可以用机器学习中的 softmax 函数来类比理解。较低的温度设置类似于较低的 softmax 温度（T），会以高确定性强调某一个偏好的结果。较高的 Gemini 温度设置则类似较高的 softmax 温度，使所选设置附近更宽范围内的结果都变得可以接受。这种不确定性的增加，适用于对精确、固定的结果要求不高的场景，例如在尝试创作型输出时。

Top-K 与 top-P

Top-K 和 top-P（也称为核采样）[4]是大语言模型中常用的两种采样设置，用于把下一个预测词元限制在预测概率最高的那部分词元之中。与温度一样，这些采样设置控制生成文本的随机性与多样性。

- **Top-K 采样**从模型预测的分布中选出概率最高的 K 个词元。top-K 越高，模型的输出越有创意、越多样；top-K 越低，模型的输出越稳定、越贴近事实。top-K 为 1 等同于贪婪解码。
- **Top-P 采样**选出累积概率不超过某个值（P）的那些概率最高的词元。P 的取值从 0（贪婪解码）到 1（大语言模型词表中的全部词元）。

要在 top-K 与 top-P 之间做选择，最好的办法是对两种方法（或两者同时使用）进行实验，看看哪种方法能得到你想要的结果。

另一项重要的配置是回答中要生成的词元数量。请注意，生成更多词元需要大语言模型进行更多计算，会导致能耗更高、响应可能更慢，进而带来更高的成本。

综合运用

如何在 top-K、top-P、温度和生成词元数量之间做选择，取决于具体的应用和期望的结果，而且这些设置会相互影响。同样重要的是，要弄清楚你所选的模型是如何把这些不同的采样设置组合在一起的。

如果温度、top-K 和 top-P 都可用（例如在 Vertex Studio 中），那么同时满足 top-K 和 top-P 条件的词元会成为下一个预测词元的候选，然后再应用温度，从通过 top-K 与 top-P 筛选的词元中采样。如果只有 top-K 或只有 top-P 可用，行为是一样的，只是仅使用其中一个 top-K 或 top-P 设置。

如果没有温度设置，那么满足 top-K 和/或 top-P 条件的词元会被随机选取，从而得到唯一的下一个预测词元。

当某项采样配置取极端值时，该项设置要么会让其他配置设置失效，要么自身变得无关紧要。

- 如果把温度设为 0，top-K 和 top-P 就变得无关紧要——概率最高的词元会成为下一个预测词元。如果把温度设得极高（大于 1，一般要到几十），温度就变得无关紧要，凡是能通过 top-K 和/或 top-P 筛选的词元都会被随机采样，以选出下一个预测词元。
- 如果把 top-K 设为 1，温度和 top-P 就变得无关紧要。只有一个词元能通过 top-K 条件，它也就是下一个预测词元。如果把 top-K 设得极高，例如等于大语言模型词表的大小，那么任何概率非零的词元都能通过 top-K 条件，不会有词元被排除。
- 如果把 top-P 设为 0（或一个很小的值），大多数大语言模型的采样实现会只考虑概率最高的那个词元来满足 top-P 条件，从而使温度和 top-K 无关紧要。如果把 top-P 设为 1，任何概率非零的词元都能满足 top-P 条件，不会有词元被排除。

作为一般性的起点，温度取 0.2、top-P 取 0.95、top-K 取 30，可以得到相对连贯、有一定创意但不过分的结果。如果你希望结果特别有创意，可以从温度 0.9、top-P 0.99、top-K 40 开始尝试。如果你希望结果不那么有创意，可以从温度 0.1、top-P 0.9、top-K 20 开始。最后，如果你的任务始终只有一个正确答案（例如解答数学题），请从温度 0 开始。

> 注意：自由度越大（温度、top-K、top-P 和输出词元数越高），大语言模型生成的文本可能越不相关。

提示技巧

大语言模型经过调优以遵循指令，并在海量数据上训练，因此能够理解提示词并生成回答。但大语言模型并不完美；你的提示词文本越清晰，大语言模型就越容易预测出接下来最可能的文本。此外，利用大语言模型的训练方式和工作原理的特定技巧，能帮助你从大语言模型那里得到相关的结果。

现在我们已经理解了什么是提示词工程、以及它需要哪些要素，接下来看一些最重要的提示技巧示例。

通用提示 / 零样本

零样本[5]提示是最简单的一类提示。它只提供任务描述和一段供大语言模型着手处理的文本。这个输入可以是任何内容：一个问题、一个故事的开头，或者一段指令。zero-shot（零样本）这个名称的含义就是「没有示例」。

我们使用 Vertex AI 中的 Vertex AI Studio（语言）[6]，它提供了一个用于测试提示词的演练场。在表 1 中，你将看到一个用于对电影评论进行分类的零样本提示示例。下文使用的表格格式是记录提示词的绝佳方式。你的提示词在最终进入代码库之前，很可能会经历许多次迭代，因此用规范、结构化的方式记录提示词工程的工作非常重要。关于这种表格格式、记录提示词工程工作的重要性以及提示词的开发流程，会在本章后文「最佳实践」一节中的「记录各种提示词尝试」中详细说明。

由于不需要创意，模型的温度应设为较低的值；我们使用 gemini-pro 的默认 top-K 和 top-P 值，它们实际上使这两项设置失效（参见上文「大语言模型的输出配置」）。请留意生成的输出。评论中 disturbing（令人不安）与 masterpiece（杰作）这两个词会让预测稍微复杂一些，因为它们出现在同一个句子里。

名称	1_1_movie_classification	
目标	将电影评论分类为正面、中性或负面。	
模型	gemini-pro	
	温度	0.1
	词元上限	5
	Top-K	N/A
	Top-P	1
提示词	将电影评论分类为 POSITIVE（正面）、NEUTRAL（中性）或 NEGATIVE（负面）。 评论：“Her” is a disturbing study revealing the direction humanity is headed if AI is allowed to keep evolving, unchecked. I wish there were more movies like this masterpiece. 情感：	
输出	POSITIVE（正面）	

表 1. 零样本提示示例

当零样本提示不奏效时，你可以在提示词中提供示范或示例，这就引出了「单样本」和「少样本」提示。

单样本与少样本

为 AI 模型编写提示词时，提供示例会很有帮助。这些示例可以帮助模型理解你的需求。当你希望把模型的输出引导到某种结构或模式上时，示例尤其有用。

单样本（one-shot）提示只提供一个示例，因此得名 one-shot。其思路是让模型有一个可以模仿的示例，从而最好地完成任务。

少样本（few-shot）提示[7]向模型提供多个示例。这种方式向模型展示了一种需要遵循的模式。其思路与单样本类似，但给出多个符合期望模式的示例，会提高模型遵循该模式的可能性。

少样本提示需要多少个示例，取决于几个因素，包括任务的复杂度、示例的质量，以及你所用生成式 AI（gen AI）模型的能力。一般来说，少样本提示至少应使用三到五个示例。不过，对于更复杂的任务，你可能需要使用更多示例；也可能因为模型的输入长度限制而需要减少示例数量。

表 2 展示了一个少样本提示示例。我们沿用之前的 gemini-pro 模型配置，只是提高了词元上限，以满足更长回答的需要。

<table>
<tr><td>目标</td><td colspan="2">将披萨订单解析为 JSON</td></tr>
<tr><td>模型</td><td colspan="2">gemini-pro</td></tr>
<tr><td></td><td>温度</td><td>0.1</td></tr>
<tr><td></td><td>词元上限</td><td>250</td></tr>
<tr><td></td><td>Top-K</td><td>N/A</td></tr>
<tr><td></td><td>Top-P</td><td>1</td></tr>
<tr><td>提示词</td><td colspan="2">将顾客的披萨订单解析为有效的 JSON：
示例：
我要一个小的披萨，加奶酪、番茄酱和意大利辣香肠。
JSON 响应：
<pre>{
  "size": "small",
  "type": "normal",
  "ingredients": [["cheese", "tomato sauce", "peperoni"]]
}</pre></td></tr>
<tr><td></td><td colspan="2">示例：
我能要一个大披萨，加番茄酱、罗勒和马苏里拉奶酪吗
<pre>{
  "size": "large",
  "type": "normal",
  "ingredients": [["tomato sauce", "bazel", "mozzarella"]]
}</pre>现在，我想要一个大披萨，其中一半加奶酪和马苏里拉奶酪，另一半加番茄酱、火腿和菠萝。
JSON 响应：</td></tr>
</table>

输出	{
	  "size": "large",
	  "type": "half-half",
	  "ingredients": [["cheese", "mozzarella"], ["tomato sauce", "ham", "pineapple"]]
	}

表 2. 少样本提示示例

在为提示词选择示例时，要使用与你想执行的任务相关的示例。示例应当多样、高质量且撰写精良。一个小错误就可能让模型产生混淆，从而导致不理想的输出。

如果你希望生成的输出能够稳健地应对各种输入，那么在示例中覆盖边缘情况就很重要。边缘情况是指那些不常见或出人意料的输入，但模型应当仍然能够处理它们。

系统提示、上下文提示与角色提示

系统提示、上下文提示和角色提示都是用来引导大语言模型生成文本的技巧，但它们关注的方面不同：

- **系统提示**为语言模型设定整体背景和目的。它定义了模型应该做什么的「大局」，例如翻译语言、对评论进行分类等。
- **上下文提示**提供与当前对话或任务相关的具体细节或背景信息。它帮助模型理解请求中的细微之处，并据此调整回答。
- **角色提示**为语言模型指定一个特定的角色或身份。这有助于模型生成与所分配角色及其相关知识、行为相一致的回答。

系统提示、上下文提示和角色提示之间可能存在相当大的重叠。例如，一个为系统指定角色的提示词，也可以带有上下文。

不过，每一类提示词的首要用途略有不同：

- **系统提示：**定义模型的基本能力和总体目的。
- **上下文提示：**提供即时的、特定于任务的信息来引导回答。它与当前任务或输入高度相关，而后者是动态变化的。
- **角色提示：**框定模型输出的风格和语气。它增加了一层具体性和个性。

区分系统提示、上下文提示和角色提示，为设计意图清晰的提示词提供了一个框架，既便于灵活组合，也让我们更容易分析每一类提示词如何影响语言模型的输出。

下面我们逐一看看这三类提示词。

系统提示

表 3 中是一个系统提示，其中额外说明了希望模型如何返回输出。我把温度调高以获得更高的创意水平，并指定了更高的词元上限。不过，由于我对如何返回输出的说明非常明确，模型并没有返回多余的文本。

目标	将电影评论分类为正面、中性或负面。

模型	gemini-pro	
	温度	1
	词元上限	5
	Top-K	40
	Top-P	0.8
提示词	将电影评论分类为 positive（正面）、neutral（中性）或 negative（负面）。只返回大写形式的标签。 评论：“Her” is a disturbing study revealing the direction humanity is headed if AI is allowed to keep evolving, unchecked. It's so disturbing I couldn't watch it. 情感：	
输出	NEGATIVE（负面）	

表 3. 系统提示示例

当你需要生成满足特定要求的输出时，系统提示会很有用。「系统提示」这个名称实际的含义是「向系统追加一项任务」。例如，你可以用系统提示生成与特定编程语言兼容的代码片段，也可以用系统提示让模型返回某种结构。请看表 4，其中我要求以 JSON 格式返回输出。

目标	将电影评论分类为正面、中性或负面，并返回 JSON。	
模型	gemini-pro	
	温度	1
	词元上限	1024
	Top-K	40
	Top-P	0.8

提示词	将电影评论分类为正面、中性或负面。返回有效的 JSON:
	评论：“Her” is a disturbing study revealing the direction humanity is headed if AI is allowed to keep evolving, unchecked. It's so disturbing I couldn't watch it.
	模式（Schema）：
	`MOVIE:`
	`{`
	`  "sentiment": String "POSITIVE" | "NEGATIVE" | "NEUTRAL",`
	`  "name": String`
	`}`
	
	`MOVIE REVIEWS:`
	`{`
	`  "movie_reviews": [MOVIE]`
	`}`
	JSON 响应：

输出	`{`
	`  "movie_reviews": [`
	`  {`
	`  "sentiment": "NEGATIVE",`
	`  "name": "Her"`
	`  }`
	`  ]`
	`}`

表 4. 使用 JSON 格式的系统提示示例

在抽取数据的提示词中要求返回 JSON 对象有一些好处。在真实应用中，我不必手动构造这种 JSON 格式，而且可以直接按排好序的顺序返回数据（处理日期时间对象时尤其方便）。但最重要的是，要求 JSON 格式会迫使模型建立一种结构，从而限制幻觉的产生。

系统提示在安全性和有害内容控制方面也非常有用。要控制输出，只需在提示词中加一句，例如：「你的回答应当保持尊重。」

角色提示

角色提示是提示词工程中的一种技巧，指的是为生成式 AI 模型指定一个特定的角色。这有助于模型生成更相关、更有信息量的输出，因为模型可以针对被赋予的角色来组织回答。

例如，你可以通过角色提示让生成式 AI 模型扮演图书编辑、幼儿园老师或励志演讲者。一旦给模型指定了角色，你就可以向它提出与该角色相关的提示。例如，你可以提示一位老师来制定一份课程教案，然后由他自己审阅。请看表 5，其中模型扮演了一名旅行向导。

目标	扮演旅行向导，提供 3 条旅行建议
模型	gemini-pro

	温度	1
	词元上限	1024
	Top-K	40
	Top-P	0.8
提示词	我希望你扮演一名旅行向导。我会告诉你我所在的地点，你来推荐附近 3 个值得去的地方。有时我也会告诉你我想去的场所类型。 我的说明：「我在阿姆斯特丹，我只想去博物馆。」 旅行建议：	
输出	1. **荷兰国家博物馆（Rijksmuseum）：**世界知名的博物馆，以伦勃朗《夜巡》等荷兰大师名作著称。不要错过那些展现荷兰艺术与历史的绘画和文物珍藏。 2. **梵高博物馆：**沉浸在文森特·梵高令人着迷的作品之中。这里收藏了他数量最多的绘画与素描，包括《星月夜》和《向日葵》。 3. **阿姆斯特丹市立博物馆：**欣赏来自世界各地的现代与当代艺术。馆藏包括毕加索、康定斯基和里特维尔德的作品，建筑本身也极具现代感。	

表 5. 角色提示示例

为 AI 模型定义一个角色视角，等于给它一份关于语气、风格和专业范围的蓝图，有助于提升输出的质量、相关性和有效性。

以下是我认为比较有效的一些风格，供你选择：

对抗式、描述式、直接、正式、幽默、有影响力、非正式、鼓舞人心、有说服力。

我们改一下表 6 中的提示词，加入幽默和鼓舞人心的风格。

提示词	我希望你扮演一名旅行向导。我会告诉你我所在的地点，请你用幽默的风格为我推荐附近 3 个值得去的地方。 我的说明：「我在曼哈顿。」 旅行建议：
输出	1. **登上「帝国之巅」：**爬上帝国大厦令人眩晕的高度，沐浴在曼哈顿天际线的荣光之中。准备好体验一把「大苹果」顶上的金刚，只不过没有大猩猩尺寸的香蕉。 2. **在 MoMA 装一回文艺范儿：**在纽约现代艺术博物馆释放你内心的艺术热情。凝视那些让你目瞪口呆的名作，甚至开始怀疑自己的火柴人画作是否还有一点艺术价值。 3. **第五大道购到手软：**在这条标志性的大道上来一场购物疗法。记得做好被价格吓到的心理准备——那些设计师精品店会让你逛橱窗（或者真的下手）时钱包哭泣。不过，你人在曼哈顿，索性就拥抱这份华丽吧！

表 6. 幽默语气与风格的角色提示示例

上下文提示

通过提供上下文提示，你可以让自己的 AI 交互尽可能顺畅、高效。模型能够更快理解你的请求，并生成更准确、更相关的回答，如表 7 的示例所示。

目标	为一个关于复古游戏的博客推荐文章选题	
模型	gemini-pro	
	温度	1
	词元上限	1024
	Top-K	40
	Top-P	0.8
提示词	上下文：你在为一个关于 80 年代复古街机游戏的博客撰稿。 请推荐 3 个可以写成文章的主题，并用几句话描述这篇文章应当包含的内容。	
输出	1. **街机框体设计的演变：**这篇文章将探讨街机框体设计的演变，从 1970 年代早期的木质和金属机箱，到 1980 年代及以后线条流畅、霓虹灯点缀的设计。 2. **来自过去的冲击：80 年代标志性街机游戏：**这篇文章将回顾 80 年代一些最具标志性、最具影响力的街机游戏的历史、影响与遗产，重点介绍它们的创新、风靡一时的玩法以及经久不衰的魅力。 3. **像素艺术的兴起与复古回潮：**这篇文章将深入探讨像素艺术作为早期街机游戏标志性视觉风格的演变过程，追溯其起源、探究其技法，并讨论像素艺术在现代游戏和数字艺术中的复兴。	

表 7. 上下文提示示例

退后一步提示

退后一步提示（step-back prompting）[8]是一种提升模型表现的技巧：先提示大语言模型思考一个与当前具体任务相关的通用问题，然后把该通用问题的答案作为后续提示的输入，用来解决具体任务。这一步「退后」让大语言模型在着手解决具体问题之前，先激活相关的背景知识和推理过程。

通过考虑更宏观、更底层的原理，大语言模型能够生成更准确、更有洞见的回答。退后一步提示鼓励大语言模型批判性地思考，并以新的、有创意的方式运用它的知识。与直接提示模型相比，这种方式会调用模型参数中更多的知识，从而改变最终执行任务的提示。

退后一步提示通过关注通用原理而非具体细节，有助于缓解大语言模型回答中的偏见。

我们来看几个示例，理解退后一步提示如何改善结果。先看一个传统的提示词（表 8），再把它与退后一步提示（表 9）做比较：

目标	为第一人称射击游戏的一个关卡写一段剧情
模型	gemini-pro

	温度	1
	词元上限	1024
	Top-K	40
	Top-P	0.8
提示词	为第一人称射击游戏的一个具有挑战性、引人入胜的新关卡写一段剧情。	
输出	关卡一开始，玩家的分队在密集的城区中遭到一支穷追不舍的敌对势力伏击。玩家必须穿过由后巷和破败建筑构成的迷宫，运用潜行与精准射击消灭敌军并避免被发现。玩家要善用环境：设置陷阱、利用掩体，并做出经过计算的行动以求生存。途中，他们发现情报显示敌人正在策划一次毁灭性袭击，玩家必须争分夺秒地收集证据，并在敌人实施计划之前把情报传回总部。	

表 8. 与退后一步提示对比之前的传统提示词

把温度设为 1 时，你可能会得到各种充满创意的剧情，但它也相当随机、平庸。所以我们退后一步：

提示词	基于流行的第一人称射击动作游戏，有哪些 5 个虚构的关键场景，能为第一人称射击游戏的关卡带来富有挑战性且引人入胜的剧情？
输出	1. **废弃军事基地：**一座庞大的后末日军事综合体，到处是变异的士兵和失控的机器人，非常适合富有挑战性的枪战。 2. **赛博朋克城市：**霓虹闪烁的未来都市环境，高楼林立、小巷密布，有经过赛博格强化的敌人和黑客机制。 3. **外星飞船：**一艘搁浅在地球上的巨大外星飞船，走廊诡异，有零重力区域，还会遇到外星生物。 4. **僵尸肆虐的小镇：**一座被成群的凶悍僵尸占据的荒凉小镇，包含激烈的近身战斗，以及寻找安全通道的解谜要素。 5. **水下研究设施：**一座被水淹没的深海实验室，充斥着变异的水生生物，需要潜行与水下探索技巧才能生存。

表 9. 为了自洽性而进行提示的示例

这些主题看起来确实很适合第一人称游戏。现在我们回到最初的提示词，只不过这次把退后一步得到的答案作为上下文一起提供，看看会返回什么。

提示词	上下文：第一人称射击游戏的 5 个引人入胜的主题： 1. **废弃军事基地：**一座庞大的后末日军事综合体，到处是变异的士兵和失控的机器人，非常适合富有挑战性的枪战。 2. **赛博朋克城市：**霓虹闪烁的未来都市环境，高楼林立、小巷密布，有经过赛博格强化的敌人和黑客机制。 3. **外星飞船：**一艘搁浅在地球上的巨大外星飞船，走廊诡异，有零重力区域，还会遇到外星生物。 4. **僵尸肆虐的小镇：**一座被成群的凶悍僵尸占据的荒凉小镇，包含激烈的近身战斗，以及寻找安全通道的解谜要素。 5. **水下研究设施：**一座被水淹没的深海实验室，充斥着变异的水生生物，需要潜行与水下探索技巧才能生存。 请从中选择一个主题，为第一人称射击游戏的一个具有挑战性、引人入胜的新关卡写一段剧情。

输出	在幽暗深渊的深处，矗立着一座破败的水下研究设施，它是人类野心及其灾难性后果的见证。它被黑暗笼罩，随着失效机械的嗡鸣而颤动，回荡着溺亡者的声音，如今已成为从深海滋生出的梦魇的温床。玩家是一名装备了先进潜水装具和实验性武器的精英海军陆战队员，被派往这片水下的恐怖领域。凭借坚韧与机智，他必须穿过危机四伏的走廊、解开隐晦的谜题，并直面潜藏在深处的可怕海怪。从抵抗深海巨大的水压，到智取狡猾的水生掠食者，在这片未知的地下世界里，每一刻都在考验人类忍耐与勇气的极限。

表 10. 为了自洽性而进行提示的示例

这看起来像一款很有意思的游戏！使用退后一步提示技巧，你可以提高提示词的准确性。

思维链（CoT）

思维链（Chain of Thought，CoT）[9]提示是一种通过生成中间推理步骤来提升大语言模型推理能力的技巧。这有助于大语言模型给出更准确的答案。你可以把思维链与少样本提示结合使用，在需要先推理再作答的复杂任务上取得更好的结果，因为零样本思维链本身是有挑战性的。

思维链有很多优点。首先，它成本很低却非常有效，而且对现成的大语言模型就管用（无需微调）。此外，思维链提示还带来了可解释性：你可以从大语言模型的回答中学习，看到它遵循的推理步骤。如果出现问题，你能够定位到问题所在。在不同的模型版本之间迁移时，思维链似乎也能提升稳健性，也就是说，相比不使用推理链的提示词，你的提示词在不同大语言模型之间的表现波动会更小。当然，它也有缺点，不过这些缺点在某种程度上是显而易见的。

大语言模型的回答中包含思维链推理，这意味着输出词元更多，也就意味着预测成本更高、耗时更长。

为了解释表 11 中的例子，我们先来写一个不使用思维链提示的提示词，展示大语言模型的缺陷。

提示词	我 3 岁的时候，我的伴侣是我年龄的 3 倍。现在，我 20 岁了。我的伴侣多大？
输出	63 岁

表 11. 尝试解决数学问题的提示词示例

哎呀，这显然是错误答案。事实上，大语言模型常常在数学任务上表现不佳，甚至会给出错误答案——哪怕只是把两个数相乘这么简单。这是因为它们是在海量文本上训练的，而数学可能需要不同的处理方式。那我们看看加入中间推理步骤是否能改善输出。

提示词	我 3 岁的时候，我的伴侣是我年龄的 3 倍。现在，我 20 岁了。我的伴侣多大？让我们一步步思考（Let's think step by step）。

输出	1. 假设我当前的年龄是「x」岁。 2. 我 3 岁的时候，年龄是 3 岁。 3. 那时我伴侣的年龄是我的 3 倍，也就是说伴侣当时是 3 × 3 = 9 岁。 4. 现在我是 20 岁，也就是说从 3 岁到现在，我的年龄增加了 20 3 = 17 岁。 5. 既然我 3 岁时伴侣 9 岁，而我的年龄增加了 17 岁，那么伴侣的年龄也一定同样增加了 17 岁。 6. 因此，我伴侣现在的年龄是 9 + 17 = 26 岁。 所以，我的伴侣 26 岁。

表 12. 思维链提示示例

很好，现在最终答案正确了。这是因为我们明确指示大语言模型解释每一步，而不是仅仅给出答案。有趣的是，模型采用的是把增加的 17 岁相加的做法。而我自己的思路是先算出伴侣与我之间相差多少岁，再加到我的年龄上（20 + (9 3)）。我们来帮模型更接近我这样的思考方式。

表 12 是「零样本」思维链的例子。思维链提示与单样本或少样本结合时会更加强大，如表 13 所示：

提示词	问：我哥哥 2 岁的时候，我的年龄是他的两倍。现在我 40 岁了。我哥哥多大？让我们一步步思考。 答：我哥哥 2 岁时，我是 2 × 2 = 4 岁。也就是说我们相差 2 岁，我更大。现在我 40 岁了，所以哥哥是 40 2 = 38 岁。答案是 38。 问：我 3 岁的时候，我的伴侣是我年龄的 3 倍。现在，我 20 岁了。我的伴侣多大？让我们一步步思考。 答：
输出	我 3 岁的时候，伴侣是 3 × 3 = 9 岁。也就是说我们相差 6 岁，伴侣更大。现在我 20 岁了，所以伴侣是 20 + 6 = 26 岁。答案是 26。

表 13. 单样本思维链提示示例

思维链可用于多种场景。比如代码生成，把需求拆解成几个步骤，并把它们对应到具体的代码行；或者在你有某种「种子」的情况下创建合成数据，例如「这个产品叫 XYZ，请根据给定标题，写出一段描述，引导模型按你会做出的假设来展开」。一般来说，任何可以通过「把过程讲出来」解决的任务，都适合使用思维链。如果你能解释解决问题的步骤，就试试思维链。

请参考 GoogleCloudPlatform GitHub 仓库中的 notebook[10]，其中更详细地介绍了思维链提示。

在本章的最佳实践部分，我们将了解一些专属于思维链提示的最佳实践。

自洽性

尽管大语言模型在各种自然语言处理任务上取得了令人瞩目的成果，但它们的推理能力常常被视为一种局限，仅靠增大模型规模无法克服。正如我们在前面思维链提示一节中学到的，可以通过提示让模型像人解题那样生成推理步骤。然而，思维链使用的是简单的「贪婪解码」策略，这限制了它的效果。自洽性[11]把采样和多数表决结合起来，生成多样的推理路径并选出最一致的答案，从而提升大语言模型回答的准确性和连贯性。自洽性给出了某个答案正确的伪概率似然，但显然成本很高。

它遵循以下步骤：

1. **生成多样的推理路径：**把同一个提示词多次提供给大语言模型。较高的温度设置会促使模型对问题生成不同的推理路径和视角。
2. 从每次生成的回答中提取答案。
3. 选择出现次数最多的答案。

我们来看一个邮件分类系统的例子，它把邮件分类为 IMPORTANT（重要）或 NOT IMPORTANT（不重要）。同一个零样本思维链提示会被多次发送给大语言模型，看看每次提交后的回答是否不同。注意邮件中友好的语气、用词以及使用的讽刺手法。所有这些都可能迷惑大语言模型。

提示词	邮件： ``` 嗨， 我看到你的网站用的是 Wordpress。一个很棒的开源内容管理 系统。我以前也用过它。它有很多出色的用户插件，而 且搭建起来相当容易。 不过我确实发现联系表单里有个 bug，当你选中姓名字段 时就会出现。请看附件截图，我在姓名字段里输入了文本。注 意我 inv0k3d 触发的那个 JavaScript 弹窗。 但除此之外，这是个很棒的网站。我很喜欢读它。你可 以放心把这个 bug 留在网站上，因为它能给我带来更多有趣 的读物。 此致， 黑客哈里。 ``` 将上述邮件分类为 IMPORTANT（重要）或 NOT IMPORTANT（不重要）。让我们一步步思考并解释原因。
输出 尝试 1	**步骤 1：确定邮件的目的。** 这封邮件的目的，是告知收件人其网站联系表单中存在一个 bug。发件人还提到他很喜欢读这个网站，并且暂时会把这个 bug 留着。 **步骤 2：评估这个 bug 可能造成的影响。** 联系表单中的 bug 可能让攻击者在网站上执行任意 JavaScript 代码。这可能导致攻击者控制网站、窃取敏感信息或篡改网站内容。 **步骤 3：考虑发件人的可信度。** 发件人自称是一名叫哈里的黑客。然而，无法核实发件人的身份。发件人可能是真心想帮忙的合法黑客，也可能是试图诱骗收件人点击恶意链接或下载恶意文件的攻击者。 **结论：IMPORTANT（重要）。** 基于该 bug 可能造成的影响以及发件人的可信度，这封邮件应归类为 **IMPORTANT（重要）**。收件人应立即采取行动修复该 bug，保护网站免遭攻击。

输出 尝试 2	**解释：** 1. **缺乏紧迫性：**邮件没有传达任何紧迫感或时间上的敏感性。它以随意的问候开头，以就事论事的语气陈述信息。 2. **并非关键 bug 报告：**联系表单中报告的 bug 被描述为有趣但并非关键。发件人甚至建议网站所有者可以把这个 bug 留着。 3. **缺乏个人影响：**邮件没有表明对网站所有者有任何直接或个人层面的影响。它只是提到发件人喜欢读这个网站。 4. **没有提出行动请求：**邮件没有明确要求网站所有者采取任何行动，没有要求修复 bug，也没有给出任何具体的改进建议。 5. **发件人的意图：**发件人的意图似乎更侧重于分享一个观察、表达对网站的兴趣，而不是提出严肃的关切或请求帮助。 **结论：NOT IMPORTANT（不重要）。**
输出 尝试 3	**步骤 1：确定邮件的目的。** 这封邮件的目的，是告知收件人其网站联系表单中存在一个 bug。发件人还提到他很喜欢读这个网站，并且暂时会把这个 bug 留着。 **步骤 2：评估这个 bug 可能造成的影响。** 联系表单中的 bug 可能让攻击者在网站上执行任意 JavaScript 代码。这可能导致攻击者控制网站、窃取敏感信息或篡改网站内容。 **步骤 3：考虑发件人的可信度。** 发件人自称是一名叫哈里的黑客。然而，无法核实发件人的身份。发件人可能是真心想帮忙的合法黑客，也可能是试图利用该 bug 谋取私利的攻击者。 **结论：** 基于上述因素，这封邮件应归类为 **IMPORTANT（重要）**。联系表单中的 bug 是严重的安全风险，而发件人的可信度未知。收件人应立即修复该 bug，并调查发件人的说法。

表 14. 为了自洽性而进行提示的示例

你可以使用上面的提示词，看看它是否会给出一致的分类结果。取决于你所使用的模型和温度配置，它可能返回「IMPORTANT（重要）」，也可能返回「NOT IMPORTANT（不重要）」。

通过生成许多条思维链，并取出现次数最多的答案（「IMPORTANT（重要）」），我们可以从大语言模型那里得到一个更稳定正确的答案。

这个例子说明，自洽性提示如何通过考虑多种视角并选择最一致的答案，来提升大语言模型回答的准确性。

思维树（ToT）

现在我们已经熟悉了思维链和自洽性提示，接下来看看思维树（Tree of Thoughts，ToT）[12]。它是对思维链提示概念的推广，因为它允许大语言模型同时探索多条不同的推理路径，而不只是沿着单一的线性思维链前进。图 1 展示了这一点。

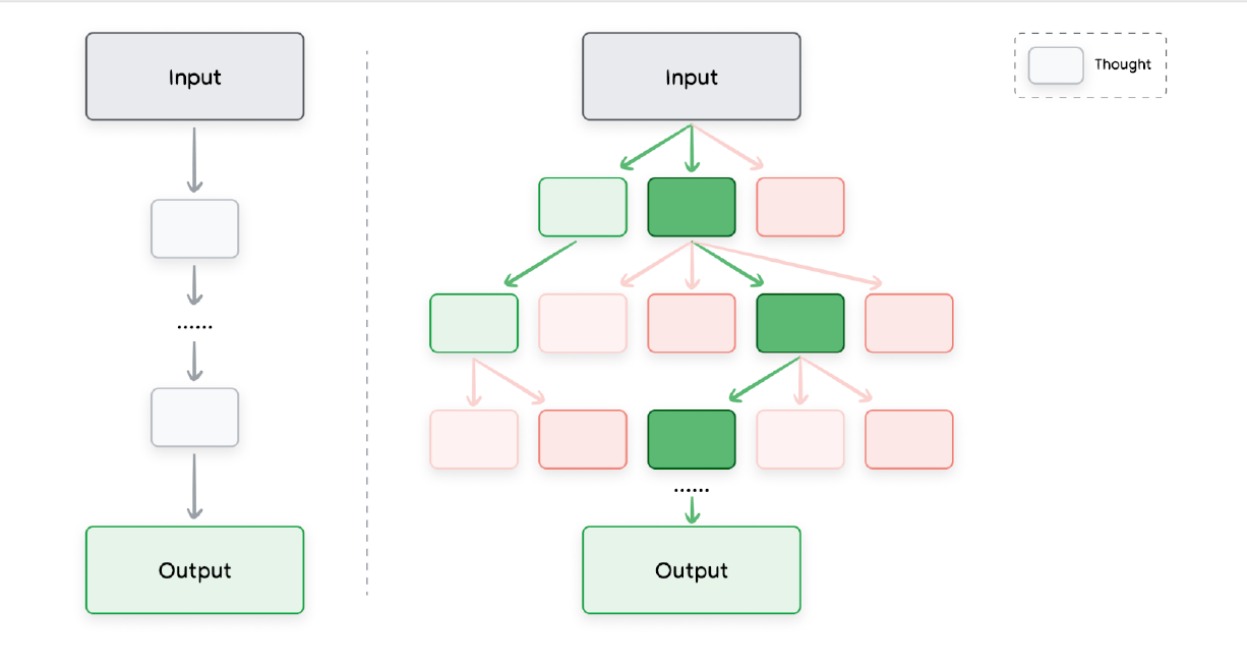


图 1. 左侧为思维链提示的示意，右侧为思维树提示的示意

这种方式使思维树特别适合需要探索的复杂任务。它的工作方式是维护一棵「思维之树」，其中每个思维都代表一段连贯的语言序列，作为解决问题的中间步骤。随后，模型可以从树中不同的节点分叉，探索不同的推理路径。

还有一个很不错的 notebook，它更详细地展示了思维树（ToT），其基础是论文《Large Language Model Guided Tree-of-Thought》。[9]

ReAct（推理与行动）

推理与行动（ReAct）[13]提示是一种让大语言模型能够解决复杂任务的范式：它把自然语言推理与外部工具（搜索、代码解释器等）结合起来，使大语言模型能够执行某些动作，例如与外部 API 交互以获取信息。这是迈向智能体建模的第一步。

ReAct 模仿了人类在现实世界中的运作方式：我们用语言推理，并且可以通过采取行动来获取信息。在各种领域里，ReAct 相比其他提示词工程方法都表现良好。

ReAct 提示的工作方式是把推理与行动组合成一个「思考—行动」循环。大语言模型首先对问题进行推理并生成行动计划，然后执行计划中的动作并观察结果。接着，大语言模型利用这些观察结果更新自己的推理，并生成新的行动计划。这个过程会一直持续，直到大语言模型得到问题的解决方案。

要实际看看它的效果，你需要写一些代码。在代码片段 1 中，我使用了 Python 的 langchain 框架，以及 VertexAI（google-cloud-aiplatform）和 google-search-results 这两个 pip 包。

要运行这个示例，你必须从 https://serpapi.com/manage-api-key 申请一个（免费的）SerpAPI 密钥，并设置环境变量 SERPAPI_API_KEY。

接下来我们写一段 Python 代码，让大语言模型去弄清楚：有多少孩子有一位在 Metallica 乐队演出的著名父亲。

Python

```
from langchain.agents import load_tools
from langchain.agents import initialize_agent
from langchain.agents import AgentType
from langchain.llms import VertexAI

prompt = "How many kids do the band members of Metallica have?"
llm = VertexAI(temperature=0.1)
tools = load_tools(["serpapi"], llm=llm)
agent = initialize_agent(tools, llm,
        agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION, verbose=True)
agent.run(prompt)
```

代码片段 1. 使用 LangChain 和 VertexAI 创建 ReAct 智能体

代码片段 2 展示了运行结果。注意 ReAct 做了一连串共五次搜索。实际上，大语言模型是在抓取 Google 搜索结果来确定乐队成员的名字。然后，它把结果列为观察，并为下一次搜索串联思考。

```
> Entering new AgentExecutor chain...
Metallica has 4 members.
Action: Search
Action Input: How many kids does James Hetfield have?
Observation: three children
Thought: 1/4 Metallica band members have 3 children
Action: Search
Action Input: How many kids does Lars Ulrich have?
Observation: 3
Thought: 2/4 Metallica band members have 6 children
Action: Search
Action Input: How many kids does Kirk Hammett have?
Observation: Hammett has been married to his second wife Lani since 1998.
They have two sons, Angel (b. September 29, 2006) and Vincenzo (b. June
28, 2008).
Thought: 3/4 Metallica band members have 8 children
Action: Search
Action Input: How many kids does Robert Trujillo have?
Observation: 2
Thought: 4/4 Metallica band members have 10 children
Final Answer: 10
```

代码片段 2. 在终端中执行 ReAct 智能体

代码片段 2 弄清楚了 Metallica 乐队有四名成员。接着它逐个搜索每位成员，询问孩子的数量并累加总数。最后，它把孩子的总数作为最终答案返回。

在实践中使用 ReAct 提示，需要理解你必须不断重新发送之前的提示与回答（并对多余生成内容进行裁剪），同时要用适当的示例与说明来设置模型。请参考 GoogleCloudPlatform GitHub 仓库中的 notebook[14]，它用一个更完整的

示例，更详细地展示了实际的大语言模型输入与输出。

自动提示词工程

走到这一步，你可能已经意识到编写提示词可以很复杂。如果能把它自动化（写一个提示词来生成提示词）岂不是很好？确实有这样一种方法：自动提示词工程（Automatic Prompt Engineering，APE）。这种方法[15]不仅免去了人工编写提示词的负担，还能提升模型在各种任务上的表现。

你会提示模型生成更多提示词，对它们进行评估，可能还会修改其中表现好的，然后重复这一过程。

例如，你可以用自动提示词工程来帮助训练一个周边 T 恤网店的聊天机器人。我们想弄清楚顾客购买乐队周边 T 恤时可能用到的各种表达方式。

1. 写出那个用来生成输出变体的提示词。在这个例子里，我使用 gemini-pro 生成 10 条指令。见表 15。

提示词	我们有一家乐队周边 T 恤网店，为了训练聊天机器人，我们需要各种不同的下单说法：「One Metallica t-shirt size S」。请生成 10 个变体，语义相同，含义保持一致。
输出	1. 我想购买一件小号的 Metallica T 恤。 2. 我可以订一件小号 Metallica T 恤吗？ 3. 我想买一件小号的 Metallica T 恤。 4. 一件 Metallica 上衣，小号，谢谢。 5. 我要一件小号的 Metallica T 恤。 6. 我想下单一件小号的 Metallica T 恤。 7. 小号 Metallica T 恤，来一件。 8. 我在找一件小号的 Metallica T 恤。 9. 你能帮我拿一件小号的 Metallica T 恤吗？ 10. 一件 Metallica 小号 T 恤，谢谢。

表 15. 用大语言模型生成提示词

2. 用选定的指标对所有候选指令打分，从而进行评估。例如可以使用 BLEU（双语评估替补，Bilingual Evaluation Understudy）或 ROUGE（面向召回的摘要评估替补，Recall-Oriented Understudy for Gisting Evaluation）。
3. 选出评估得分最高的候选指令。这个候选就会成为你可以在软件应用或聊天机器人中使用的最终提示词。你也可以对选出的提示词再做调整，并重新评估。

代码提示

Gemini 主要面向基于文本的提示词，其中也包括为返回代码而编写的提示词。我们到 Vertex AI Studio 里测试这些提示词，看一些编程示例。

用于编写代码的提示词

Gemini 也可以充当开发者，帮你用任意编程语言编写代码。作为开发者，这能帮助你加快写代码的速度。

设想你机器上有一个文件夹，里面有几百个文件需要重命名。逐个重命名会耗费大量时间。你会一点 Bash，可以写个脚本来自动完成，但这也需要花些时间。那我们就写一个提示词吧。你可以在面向公众的 Gemini 聊天机器人里编写这个提示词；如果你更在意保密性，也可以在自己的 Google Cloud 账号中打开 Vertex AI Studio 来编写。Vertex AI Studio 的优势在于你可以配置温度等参数。

名称	1_bash_rename_files_in_folder	
目标	编写一个提示词，让模型用 Bash 写代码来重命名文件夹中的文件。	
模型	gemini-pro	
	温度	0.1
	词元上限	1024
	Top-K	N/A
	Top-P	1
提示词	用 Bash 写一段代码片段，要求输入一个文件夹名称，然后获取该文件夹的内容，并给其中所有文件重命名，在文件名前面加上 draft。	

输出	```bash
	#!/bin/bash
	# Ask for the folder name
	echo "Enter the folder name: "
	read folder_name
	# Check if the folder exists
	if [! -d "$folder_name"]; then
	  echo "Folder does not exist."
	  exit 1
	fi
	# Get the list of files in the folder
	files=("$folder_name"/*)
	# Rename each file by prepending "draft" to the file name
	for file in "${files[@]}"; do
	  new_file_name="draft_$(basename "$file")"
	  mv "$file" "$new_file_name"
	done
	echo "Files renamed successfully."
	``	

表 16. 使用 Gemini 编写代码

在我看来这段代码写得不错——甚至还带有文档注释！不过，由于大语言模型无法真正推理，只是重复训练数据，因此必须先阅读并测试代码。

大家最期待的时刻到了：它真的能跑通吗？

我们先用一个只有少量文件的测试文件夹试一试，这些文件需要从 filename.txt 重命名为 draft_filename.txt。

1. 复制表 16 的输出（去掉 ```bash ``` 文本包装），粘贴到一个新文件里，命名为 「rename_files.sh」。
2. 打开终端窗口，输入：. rename_files.sh。它会要求你输入文件夹名称，例如 test，然后按回车。
3. 脚本似乎运行正常。你会看到消息：Files renamed successfully。查看 test 文件夹时，注意所有文件都完美地重命名成了 draft_filename.txt。

它奏效了！

用于解释代码的提示词

作为开发者，在团队中工作时你必须阅读别人写的代码。Gemini 在这方面也能帮上忙。我们把表 16 的代码输出拿过来，去掉注释，然后请大语言模型解释它做了什么，见表 17：

名称	1_bash_explain_the_code	
目标	编写一个提示词来解释 Bash 代码。	
模型	gemini-pro	
	温度	0.1
	词元上限	1024
	Top-K	N/A
	Top-P	1
提示词	请向我解释下面的 Bash 代码： ``` #!/bin/bash echo "Enter the folder name: " read folder_name if [! -d "$folder_name"]; then echo "Folder does not exist." exit 1 fi files=("$folder_name"/*) for file in "${files[@]}"; do new_file_name="draft_$(basename "$file")" mv "$file" "$new_file_name" done echo "Files renamed successfully." ```	

输出	这段 Bash 脚本执行以下任务： 1. **用户输入：** 它使用 `echo` 和 `read` 命令提示用户输入文件夹名称。输入的文件夹名称保存在 `folder_name` 变量中。 2. **检查文件夹是否存在：** 它使用 `[ ! -d "$folder_name" ]` 条件检查指定名称的文件夹是否存在。如果文件夹不存在，就显示错误消息（“Folder does not exist.”）并以退出码 1 结束脚本。 3. **列出文件：** 它使用 `files=( "$folder_name"/* )` 命令列出指定文件夹内的所有文件。这会创建一个名为 `files` 的数组，其中包含该文件夹中所有文件的路径。 4. **重命名文件：** • 它用 `for` 循环遍历 `files` 数组中的每个文件。 • 对每个文件，它用 `new_file_name="draft_$(basename "$file")"` 命令生成新的文件名。这个新文件名是在原文件名前加上字符串 “draft_” 得到的。 • 然后它用 `mv "$file" "$new_file_name"` 命令重命名文件，也就是把文件移动到新的文件名下。 5. **成功消息：** • 重命名完所有文件后，它显示成功消息（“Files renamed successfully.”），表示重命名操作成功。

表 17. 用于解释代码的提示词

用于翻译代码的提示词

表 16 中的 bash 代码似乎运行正常。不过，如果这个脚本能提示我输入文件名，它其实很有复用价值，理想情况下它应该作为一个带用户界面的独立应用来工作。作为起点，对于（Web）应用来说 Python 比 Bash 更合适。大语言模型可以帮助把代码从一种语言翻译成另一种语言。见表 18 的例子：

名称	1_python_rename_files	
目标	编写一个提示词，把 Bash 代码翻译成 Python。	
模型	gemini-pro	
	温度	0.1
	词元上限	1024
	Top-K	N/A
	Top-P	1

提示词	把下面的 Bash 代码翻译成一段 Python 代码。
	``bash
	#!/bin/bash
	echo "Enter the folder name: "
	read folder_name
	if [! -d "$folder_name"]; then
	  echo "Folder does not exist."
	  exit 1
	fi
	files=("$folder_name"/*)
	for file in "${files[@]}"; do
	  new_file_name="draft_$(basename "$file")"
	  mv "$file" "$new_file_name"
	done
	echo "Files renamed successfully."
	``

输出	```python
	import os
	import shutil
	
	# Get the folder name from the user
	folder_name = input("Enter the folder name: ")
	
	# Check if the folder exists
	if not os.path.isdir(folder_name):
	  print("Folder does not exist.")
	  exit(1)
	
	# Get a list of all files in the folder
	files = os.listdir(folder_name)
	
	# Iterate over the files
	for file in files:
	  # Create a new file name with the prefix "draft_"
	  new_file_name = f"draft_{file}"
	  # Move the file to the new name
	  shutil.move(os.path.join(folder_name, file),
	  os.path.join(folder_name, new_file_name))
	
	# Print a success message
	print("Files renamed successfully.")
	``

表 18. 把代码从 Bash 翻译成 Python 的提示词

请阅读并检查代码。复制提示词的输出，粘贴到一个新文件：file_renamer.py。打开终端窗口，执行命令 python file_renamer.py 来测试代码。

注意：在 Vertex AI 的语言工作室（Language Studio）中提示生成（Python）代码时，你需要点击「Markdown」按钮。否则你会得到纯文本，行首缩进会丢失，而这对运行 Python 代码很重要。

用于调试和审查代码的提示词

我们手动给表 18 的代码写几处修改。它应该提示用户输入文件名前缀，并把这个前缀以大写字符写入。请看代码片段 3 中的示例代码，但糟糕的是，它现在返回 Python 错误！

Python

```
import os
import shutil

folder_name = input("Enter the folder name: ")
prefix = input("Enter the string to prepend to the filename: ")
text = toUpperCase(prefix)

if not os.path.isdir(folder_name):
  print("Folder does not exist.")
  exit(1)

files = os.listdir(folder_name)
for file in files:
  new_filename = f"{text}_{file}"
  shutil.move(os.path.join(folder_name, file),
              os.path.join(folder_name, new_file_name))

print("Files renamed successfully.")
```

代码片段 3. 一段有问题的 Python 脚本

哎呀！这看起来是个 bug：

```
The below Python code gives an error:
Traceback (most recent call last):
  File "/Users/leeboonstra/Documents/test_folder/rename_files.py", line 7,
in <module>
    text = toUpperCase(prefix)
NameError: name 'toUpperCase' is not defined
```

代码片段 4. 我把 Python 代码改坏了

我们看看能不能让大语言模型来调试和审查代码。请看表 19：

名称	1_python_debug_code
目标	编写一个提示词来调试和审查 Python 代码。
模型	gemini-pro

	温度	0.1
	词元上限	1024
	Top-K	N/A
	Top-P	1

提示词	下面的 Python 代码报错:
	`Traceback (most recent call last):`
	`  File "/Users/leeboonstra/Documents/test_folder/rename_files.py",`
	`line 7, in <module>`
	`  text = toUpperCase(prefix)`
	`NameError: name 'toUpperCase' is not defined`
	请调试问题出在哪里，并说明我可以如何改进这段代码。
	``python
	import os
	import shutil
	
	# Get the folder name from the user
	folder_name = input("Enter the folder name: ")
	prefix = input("Enter the string to prepend to the filename: ")
	text = toUpperCase(prefix)
	
	# Check if the folder exists
	if not os.path.isdir(folder_name):
	  print("Folder does not exist.")
	  exit(1)
	
	# Get a list of all files in the folder
	files = os.listdir(folder_name)
	
	# Iterate over the files
	for file in files:
	  # Create a new file name with the prefix "draft_"
	  new_filename = f"{text}_{file}"
	  # Move the file to the new name
	  shutil.move(os.path.join(folder_name, file),
	  os.path.join(folder_name, new_file_name))
	
	# Print a success message
	print("Files renamed successfully.")
	``	

输出	这段代码调用了 `toUpperCase` 函数来把 `prefix` 转换为大写，但该函数并未定义。要解决这个问题，你可以使用字符串类的 `upper()` 方法，它会把给定字符串转换为大写。修改后的代码如下：
	``python
	import os
	import shutil
	
	# Get the folder name from the user
	folder_name = input("Enter the folder name: ")
	prefix = input("Enter the string to prepend to the filename: ")
	text = prefix.upper()
	
	# Check if the folder exists
	if not os.path.isdir(folder_name):
	  print("Folder does not exist.")
	  exit(1)
	
	# Get a list of all files in the folder
	files = os.listdir(folder_name)
	
	# Iterate over the files
	for file in files:
	  # Create a new file name with the prefix "draft_"
	  new_filename = f"{text}_{file}"
	  # Move the file to the new name
	  shutil.move(os.path.join(folder_name, file),
	              os.path.join(folder_name, new_filename))
	
	# Print a success message
	print("Files renamed successfully.")
	``

此外，这段代码还可以做几处其他改进：

1. 新文件名没有保留文件的扩展名。更好的做法是从原文件名中提取扩展名，并把它追加到新文件名上。
2. 代码假定文件夹名称中不含空格。不过，把空格替换成下划线或连字符来优雅处理空格是一种好做法。
3. 使用 `f` 字符串语法进行字符串插值，比传统的 `+` 运算符更易读、更简洁。
4. 代码没有处理重命名过程中可能出现的错误。更好的做法是把 `shutil.move` 调用放在 `try...except` 块中，以捕获潜在错误。

下面是采纳这些建议后的改进代码：

````
```python
import os
import shutil

# Get the folder name from the user
folder_name = input("Enter the folder name: ")
prefix = input("Enter the string to prepend to the filename: ")

# Check if the folder exists
if not os.path.isdir(folder_name):
  print("Folder does not exist.")
  exit(1)

# Get a list of all files in the folder
files = os.listdir(folder_name)

# Iterate over the files
for file in files:
  # Get the file extension
  ext = os.path.splitext(file)[1]
  # Create a new file name with the prefix and the file extension
  new_filename = f"{prefix}_{os.path.splitext(file)[0]}{ext}"
  # Build the full paths for the original and new files
  old_path = os.path.join(folder_name, file)
  new_path = os.path.join(folder_name, new_filename)
  # Try to move the file
  try:
    shutil.move(old_path, new_path)
  except Exception as e:
    print(f"An error occurred while moving the file {file}: {e}")

# Print a success message
print("Files renamed successfully.")
```
````

这些改进使代码更健壮、更灵活，同时也更易于阅读和理解……（由于达到了词元上限，回答被截断了。如果你需要更长的回答，请尝试提高词元上限。）

表 19. 用于调试和审查 Python 代码的提示词

太棒了。它不仅告诉我如何解决这个问题，还发现我的代码里还有别的 bug，并给出了解决办法。提示词的最后一部分还给出了整体改进代码的建议。

多模态提示呢?

编写代码的提示仍然使用同一个常规的大语言模型。多模态提示是另一个话题，它指的是一种使用多种输入格式来引导大语言模型的技术，而不仅仅依赖文本。根据模型的能力和手头的任务，它可以是文本、图像、音频、代码，甚至其他格式的组合。

最佳实践

找到合适的提示词需要反复调试。Vertex AI 中的语言工作室是试玩提示词的理想场所，你可以在那里针对不同模型进行测试。

遵循以下最佳实践，你就能成为提示词工程的高手。

提供示例

最重要的一条最佳实践，就是在提示词中提供（单样本 / 少样本）示例。这非常有效，因为它是一种强有力的教学手段。这些示例展示了期望的输出或类似的回答，让模型能够从中学习并据此调整自己的生成。这相当于给模型一个参照点或目标，使它的回答在准确性、风格和语气上更贴近你的预期。

简洁设计

提示词应当简洁、清晰，对你自己和模型来说都容易理解。一般来说，如果它已经把你弄糊涂了，那多半也会把模型弄糊涂。尽量不要使用复杂的语言，也不要提供不必要的信息。

示例：

修改前：

```
我现在正在纽约旅行，很想多了解一些好去处。我带着两个 3 岁的孩子。
假期里我们应该去哪里？
```

改写后：

```
扮演一名面向游客的旅行向导。请描述纽约曼哈顿适合带 3 岁孩子游玩的好地方。
```

尽量使用描述动作的动词。以下是一组示例：

扮演、分析、归类、分类、对比、比较、创建、描述、定义、评估、提取、查找、生成、识别、列举、衡量、组织、解析、挑选、预测、提供、排名、推荐、返回、检索、改写、选择、展示、排序、总结、翻译、撰写。

明确输出的具体要求

要明确说明你期望的输出。过于简短的指令可能不足以引导大语言模型，或者过于笼统。在提示词中（通过系统提示或上下文提示）给出具体的细节，可以帮助模型聚焦于相关的内容，从而提升整体的准确性。

示例：

要做：

```
写一篇 3 段式的博客文章，介绍排名前 5 的游戏主机。文章应当信息丰富、引人入胜，
并以对话式的风格撰写。
```

不要做：

```
写一篇关于游戏主机的博客文章。
```

优先使用指令，而非约束

在提示词中，指令和约束都用来引导大语言模型的输出。

- **指令**明确说明期望回答的格式、风格或内容。它引导模型应该做什么、产出什么。
- **约束**是对回答的一系列限制或边界。它限定模型不应该做什么、应当避免什么。

越来越多的研究表明，在提示词中关注正向指令，可能比大量依赖约束更有效。这种做法也符合人类更偏好正向指令、而非一堆「不要做什么」清单的习惯。

指令直接传达期望的结果，而约束可能让模型去猜测什么才是被允许的。指令在既定边界内给模型灵活性并鼓励创造力，而约束则可能限制模型的潜力。此外，一长串约束还可能相互冲突。

在某些情况下，约束仍然很有价值，例如防止模型生成有害或带偏见的内容，或者需要严格的输出格式或风格时。

如果可能，请使用正向指令：与其告诉模型不要做什么，不如告诉它应该做什么。这样可以避免混淆，并提升输出的准确性。

要做：

```
写一篇 1 段式的博客文章，介绍排名前 5 的游戏主机。只讨论主机本身、制造商、
发行年份和总销量。
```

不要做：

```
写一篇 1 段式的博客文章，介绍排名前 5 的游戏主机。不要列游戏名称。
```

作为最佳实践，请从优先使用指令开始，清楚地说明你希望模型做什么，只在出于安全、清晰或特定要求的需要时才使用约束。

通过实验和迭代，测试指令与约束的不同组合，找出最适合你具体任务的做法，并把它们记录下来。

控制最大词元长度

要控制大语言模型生成回答的长度，你可以在配置中设置最大词元上限，或者在提示词中明确要求某个具体长度。例如：

```
「用一条推文长度的消息解释量子物理。」
```

在提示词中使用变量

为了复用提示词并让它更动态，可以在提示词中使用变量，针对不同的输入替换变量的取值。例如，如表 20 所示，这是一个提供城市相关事实的提示词。与其把城市名写死在提示词里，不如使用变量。变量可以让你避免重复，从而节省时间和精力。如果你需要在多个提示词中使用同一份信息，可以把它存到一个变量里，然后在每个提示词中引用该变量。在把提示词集成到自己的应用中时，这尤其有用。

提示词	**变量** {city} = "Amsterdam" **提示词** 你是一名旅行向导。告诉我关于这座城市的一个事实：{city}
输出	阿姆斯特丹是一座美丽的城市，遍布运河、桥梁和狭窄的街道。它历史悠久、文化丰富、夜生活精彩，非常值得一游。

表 20. 在提示词中使用变量

尝试不同的输入格式和写作风格

不同的模型、模型配置、提示词格式、用词以及提交方式，都会带来不同的结果。因此，实验不同的提示词属性很重要，例如风格、用词以及提示词类型（零样本、少样本、系统提示）。

举例来说，一个旨在生成关于革命性游戏主机 Sega Dreamcast 的文本的提示词，可以写成问题、陈述或指令，得到的结果也会不同：

- **问题：**Sega Dreamcast 是什么，为什么它是一台如此具有革命性的主机？
- **陈述：**Sega Dreamcast 是世嘉在 1999 年发布的第六代游戏主机。它……
- **指令：**写一段文字，介绍 Sega Dreamcast 主机，并解释它为什么如此具有革命性。

少样本分类任务要打乱类别顺序

一般来说，少样本示例的顺序不应该有太大影响。不过，在做分类任务时，请务必打乱少样本示例中可能出现的各类别。否则，你可能会过拟合到示例的特定顺序上。通过打乱可能出现的类别，你可以确保模型学到的是识别每个类别的关键特征，而不是简单地记住示例的顺序。这样在未见过的数据上会得到更稳健、更具泛化能力的表现。

一个经验法则是：从 6 个少样本示例开始，并以此为基础测试准确率。

适应模型更新

你需要及时了解模型架构的变化、新增的数据和能力。尝试更新的模型版本，并调整你的提示词，以更好地利用新的模型特性。像 Vertex AI Studio 这样的工具非常适合存储、测试和记录提示词的各个版本。

尝试不同的输出格式

除了提示词的输入格式，也可以考虑尝试不同的输出格式。对于提取、选择、解析、排序、排名或分类数据这类非创作型任务，可以尝试让输出以 JSON 或 XML 这类结构化格式返回。

在抽取数据的提示词中要求返回 JSON 对象有一些好处。在真实应用中，我不必手动构造这种 JSON 格式，而且可以直接按排好序的顺序返回数据（处理日期时间对象时尤其方便）。但最重要的是，要求 JSON 格式会迫使模型建立一种结构，从而限制幻觉的产生。

少样本提示一节中的表 4 展示了一个如何返回结构化输出的示例。

与其他提示词工程师一起实验

如果你遇到一个必须想办法写出好提示词的场景，不妨找几个人各自尝试一次。当每个人都遵循本章列出的最佳实践时，你会看到不同提示词尝试之间的表现存在差异。

思维链的最佳实践

对于思维链提示，把答案放在推理之后是必需的，因为推理的生成会改变模型在预测最终答案时所获得的词元。

使用思维链和自洽性时，你需要能够从提示词的输出中把最终答案与推理过程分开提取出来。

对于思维链提示，请把温度设为 0。

思维链提示基于贪婪解码，也就是根据语言模型赋予的最高概率来预测序列中的下一个词。一般来说，在使用推理得出最终答案时，很可能只有一个正确答案。因此温度应当始终设为 0。

记录各种提示词尝试

最后这条建议在本章前面已经提到过，但怎么强调它的重要性都不为过：请尽可能详细地记录你的提示词尝试，这样你才能随着时间推移了解哪些做法有效、哪些无效。

提示词的输出可能因模型不同、采样设置不同，甚至同一模型的不同版本而不同。而且，即使是发给同一模型的完全相同的提示词，输出在句子格式和用词上也可能出现细微差异。（例如前面提到的，如果两个词元具有相同的预测概率，平局可能是随机打破的，这又会影响到后续预测的词元。）

我们建议按照表 21 的模板创建一个 Google 表格。这种做法的好处是：当你不可避免地要回顾自己的提示词工作时——无论是将来重新拾起它（你会惊讶于仅仅休息一小段时间后就能忘掉多少东西）、测试提示词在不同模型版本上的表现，还是帮助调试未来的错误——你都有一份完整的记录。

除了表中的字段之外，还建议记录提示词的版本（迭代次数）、一个用于标记结果属于 OK / 不 OK / 有时 OK 的字段，以及一个用于记录反馈的字段。如果你有幸使用 Vertex AI Studio，请保存你的提示词（使用与文档记录中相同的名称和版本），并在表格中记录指向已保存提示词的超链接。这样，你距离重新运行提示词永远只有一次点击之遥。

在开发检索增强生成（RAG）系统时，你还需要记录 RAG 系统中影响哪些内容被插入提示词的具体方面，包括查询、分块设置、分块输出以及其他信息。

名称	[提示词的名称和版本]	
目标	[用一句话说明这次尝试的目标]	
模型	[所用模型的名称和版本]	
	温度	[0–1 之间的取值]
	词元上限	[数值]
	Top-K	[数值]
	Top-P	[数值]
提示词	[写出完整的提示词]	
输出	[写出输出结果，或多次输出的结果]	

表 21. 用于记录提示词的模板

当你觉得提示词已经接近完美时，把它带入你的项目代码库。在代码库中，请把提示词与代码分开存放在单独的文件里，这样更便于维护。最后，理想情况下你的提示词应当是一个已投入运营的系统的一部分，作为提示词工程师，你应当依靠自动化测试和评估流程来了解你的提示词在某个任务上的泛化能力究竟如何。

提示词工程是一个迭代的过程。设计并测试不同的提示词，分析并记录结果，根据模型的表现改进提示词。持续实验，直到获得期望的输出。当你更换模型或模型配置时，请回过头来继续用之前用过的提示词做实验。

总结

本白皮书讨论了提示词工程。我们学习了各种提示技巧，例如：

- 零样本提示
- 少样本提示
- 系统提示
- 角色提示
- 上下文提示
- 退后一步提示
- 思维链
- 自洽性

- 思维树
- ReAct

我们还了解了如何实现提示词的自动化。

随后，白皮书讨论了生成式 AI 带来的挑战，例如提示词不充分时可能出现的各种问题。最后，我们以成为更优秀的提示词工程师的最佳实践作结。

尾注

1. Google，2023，Gemini by Google。见：https://gemini.google.com。
2. Google，2024，Gemini for Google Workspace Prompt Guide。见：https://inthecloud.withgoogle.com/gemini-for-google-workspace-prompt-guide/dl-cd.html。
3. Google Cloud，2023，Introduction to Prompting。见：https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/introduction-prompt-design。
4. Google Cloud，2023，Text Model Request Body: Top-P & top-K sampling methods。见：https://cloud.google.com/vertex-ai/docs/generative-ai/model-reference/text#request_body。
5. Wei, J., et al.，2023，Zero Shot - Fine Tuned language models are zero shot learners。见：https://arxiv.org/pdf/2109.01652.pdf。
6. Google Cloud，2023，Google Cloud Model Garden。见：https://cloud.google.com/model-garden。
7. Brown, T., et al.，2023，Few Shot - Language Models are Few Shot learners。见：https://arxiv.org/pdf/2005.14165.pdf。
8. Zheng, L., et al.，2023，Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models。见：https://openreview.net/pdf?id=3bq3jsvcQ1。
9. Wei, J., et al.，2023，Chain of Thought Prompting。见：https://arxiv.org/pdf/2201.11903.pdf。
10. Google Cloud Platform，2023，Chain of Thought and React。见：https://github.com/GoogleCloudPlatform/generative-ai/blob/main/language/prompts/examples/chain_of_thought_react.ipynb。
11. Wang, X., et al.，2023，Self Consistency Improves Chain of Thought reasoning in language models。见：https://arxiv.org/pdf/2203.11171.pdf。
12. Yao, S., et al.，2023，Tree of Thoughts: Deliberate Problem Solving with Large Language Models。见：https://arxiv.org/pdf/2305.10601.pdf。
13. Yao, S., et al.，2023，ReAct: Synergizing Reasoning and Acting in Language Models。见：https://arxiv.org/pdf/2210.03629.pdf。
14. Google Cloud Platform，2023，Advance Prompting: Chain of Thought and React。见：https://github.com/GoogleCloudPlatform/applied-ai-engineering-samples/blob/main/genai-on-vertex-ai/advanced_prompting_training/cot_react.ipynb。
15. Zhou, C., et al.，2023，Automatic Prompt Engineering - Large Language Models are Human-Level Prompt Engineers。见：https://arxiv.org/pdf/2211.01910.pdf。